

Cambios realizados respecto a la rama main:

- **punto 1: Hacer el código necesario para gestionar los argumentos que se le pasan al programa.**

MODIFICACIONES

1. Tamaño de memoria compartida dinámico

Versión original:

```
shmget(1234, 298220, IPC_CREAT | 0666);
```

Versión corregida:

```
int tam = PARKING_getTamaNoMemoriaCompartida();  
shmget(IPC_PRIVATE, tam + sizeof(MEM_PROPIA), IPC_CREAT | 0600);
```

El valor 298220, la biblioteca puede cambiar ese tamaño entre versiones o configuraciones. Si el tamaño reservado es menor del necesario, la biblioteca escribirá fuera de la memoria asignada, corrompiendo datos o causando un fallo de segmentación (segfault). La función `PARKING_getTamaNoMemoriaCompartida()` existe precisamente para que el programa siempre pida el tamaño correcto.

2. Número de semáforos dinámico

Versión original:

```
semget(5678, 10, IPC_CREAT | 0666);
```

Versión corregida:

```
int nSem = PARKING_getNSemAforos();  
semget(IPC_PRIVATE, nSem + NUM_SEM_PROPIOS, IPC_CREAT | 0600);
```

Un poco como antes, si se crean menos semáforos de los que la biblioteca necesita (en este caso teníamos 10), cualquier operación `semop` sobre un índice inexistente

fallará con error. La función `PARKING_getNSemAforos()` devuelve el número exacto requerido, así evitamos asumir un valor que puede ser incorrecto.

3. Claves IPC fijas a `IPC_PRIVATE`

Versión original:

```
shmget(1234, ...)
semget(5678, ...)
```

Versión corregida:

```
shmget(IPC_PRIVATE, ...)
semget(IPC_PRIVATE, ...)
```

Las claves numéricas fijas como 1234 y 5678 son globales al sistema. Si otro proceso, o una ejecución anterior que no limpió bien sus recursos, ya creó recursos con esas claves, `shmget` y `semget` devolverán el ID de ese recurso, con tamaño y permisos incorrectos. Con `IPC_PRIVATE` siempre creamos un recurso nuevo y exclusivo para el proceso actual.

4. Permisos 0666 a 0600

Práctica más correcta y segura, solo el propio usuario puede leer y escribir. De todas maneras no se hasta que punto aquí puede marcar algún tipo de diferencia.

5. Buzón de mensajes creado correctamente

Versión original:

```
PARKING_inicio(..., 0, ...)
```

Versión corregida:

```
int id_buzon = msgget(IPC_PRIVATE, IPC_CREAT | 0600);
PARKING_inicio(..., id_buzon, ...)
```

PARKING_inicio recibe el ID del buzón como parámetro y lo usa internamente para la comunicación entre procesos (entre el gestor y los chóferes). 0 no es un ID válido, ya que podría corresponder a un recurso del sistema o simplemente no existir, acabando en fallos. Es necesario crear el buzón con msgget y pasarle su ID real.

6. Función de llegada: int f(int) a int f(HCoche)

Versión original:

```
int mi_llegada_prueba(int id) { ... }
```

Versión corregida:

```
int mi_llegada_prueba(HCoche hc) { ... }
```

TIPO_FUNCION_LLEGADA está definido en parking.h como un puntero a función que recibe HCoche hc.

7. Limpieza de recursos en una función

```
void limpiar(void) {  
    if (mem_base != NULL && mem_base != (char *)-1) shmdt(mem_base);  
    if (id_mem != -1) shmctl(id_mem, IPC_RMID, NULL);  
    if (id_sem != -1) semctl(id_sem, 0, IPC_RMID);  
    if (id_buzon != -1) msgctl(id_buzon, IPC_RMID, NULL);  
} //fin funcion limpiar
```

En una función porque luego si la creación de algún recurso falla, como el buzón, habría que forzar a eliminarlo, por tanto habría que duplicar en el código la liberación de recursos.

(Ejemplo:

Memoria compartida creada

Conjunto de semáforos creado (ID: 0)

Buzón de mensajes falla → Habría que liberar los 3, el que falla y el que no)

Y lo mismo si fallasen lo semáforos. Más luego al final del programa liberar todo... es mucho más limpio usar una función para eso (la explico mas adelante en añadidos).

AÑADIDOS

1. Función para gestión de argumentos

La función `validar_argumentos` valida que, se introduzcan minimo 3 argumentos y máximo 5 (`./programa ret chof` como minimo y `./programa ret chof D PA` (o PD en su defecto) como máximo).

Despues, a partir del argumento `chof`, identifica si la cadena introducida corresponde a D, PA o PD para comprobar si se introdujeron los argumentos correctos. Finalmente comprueba si PA y PD se introdujeron juntos, lo cual no es válido.

2. Variables globales

```
int retardo, num_choferes, debug, prio_PA, prio_PD;  
int id_mem = -1, id_sem = -1, id_buzon = -1;
```

Para que sea más sencillo gestionar luego el paso de parámetros a los diferentes procesos. Lo suyo sería crear un struct con todos ellos pero de momento lo dejo así, al final no es difícil cambiarlo después.

Las tres del final las inicio en -1 por que asi se puede saber que no se crearon todavía esos recursos.

- **punto 2: Crear los recursos IPC necesarios (semáforos, memoria compartida, buzón de paso de mensajes), y comprobad que se crean bien, con ipcs. Es preferible, para que no haya interferencias, que los defináis privados.**

1. DEFINE

```
#define NUM_ALGORITMOS 4
```

```
#define NUM_SEM_PROPIOS (2 + NUM_ALGORITMOS)
```

```
#define TAM_PARKING 80
```

- **NUM_ALGORITMOS:** Los cuatro algoritmos de los cuatro parkings (primer, siguiente, mejor, peor). Para tema de dimensionar arrays.
- **NUM_SEMS_PROPIOS:** Número total de semáforos que necesita nuestro propio programa (1 chófer + 1 mutex + 4 de orden). Estos se suman a los de la biblioteca. Sumo NUM_ALGORITMOS por que realmente de ahí se sacan los 4 semaforos extra de orden, pero se podría poner 6 directamente en lugar de 1 + NUM_ALGORITMOS
- **TAM_PARKING:** Para dimensionar el array que contiene las posiciones de cada acera, que tiene 80 posiciones (de 0 a 79).

```
#define IDX_SEM_CHOFER (nSem + 0)
```

```
#define IDX_SEM_MUTEX (nSem + 1)
```

```
#define IDX_SEM_ORDEN(a) (nSem + 2 + (a))
```

Estos calculan el índice real de cada semáforo propio dentro del array de semáforos.

Los primeros nSem índices están reservados para la biblioteca, los semaforos propios empiezan a partir de ahí.

Si la biblioteca tiene 10 semaforos, nSem = 10 y array[0...9] (es decir, ocupan hasta la posición nSem - 1). Por tanto, el siguiente semáforo, que sería el primer semáforo propio, sería el del índice nSem. Pasaríamos de array[0...9] a array[0...10]. Los demás aplican esa misma lógica.

El último gestiona el orden en que aparcen los coches en cada algoritmo. Varía entre 0 y 1. Un chofer coge el coche 1 y busca aparcamiento, SEM_ORDEN vale 0 y bloquea a más procesos de aparcar. Cuando el coche 1 aparca, SEM_ORDEN pasa a valer 1 y por tanto

libera al siguiente proceso (siguiente chofer con el coche 2) y mientras aparca vuelve SEM_ORDEN a valer 0. (esto esta por implementar, pero creo que seria algo asi, se ira viendo).

2. Struct

```
typedef struct {  
    int acera[NUM_ALGORITMOS][TAM_PARKING];  
    int proxAparcar[NUM_ALGORITMOS];  
    int ultimoAparcado;  
    int terminar;  
    int n_aparcar;  
    int n_desaparcar;  
} MEM_PROPIA;
```

Agrupar todos los datos que los procesos hijos necesitan compartir entre sí para tema memoria compartida.

- **acera[][]**: Matriz de 4 aceras × 80 posiciones. Cada posición vale 0 si está libre o el tamaño del coche que la ocupa. Se consulta para comprobar el espacio libre en la acera cuando un coche quiere aparcar.
- **proxAparcar[]**: Número del próximo coche que debe aparcar en cada algoritmo. Garantiza que los coches aparquen en orden numérico.
- **ultimoAparcado**: Posición donde aparcó el último coche. Solo sirve para el algoritmo de siguiente ajuste, que busca a partir de esa posición.
- **terminar**: Flag que el padre pone a 1 cuando la simulación tiene que terminar. Los chóferes lo comprueban en su bucle para saber cuándo salir.
- **n_aparcar y n_desaparcar**: Contadores de mensajes pendientes de aparcar y desaparcar. Los usa el gestor para implementar el tema de PA (prioridad para aparcar) y PD (prioridad para desaparcar).

3. Más Variables Globales

```
int nSem = 0, tam = 0;
```

Almacenan los tamaños que devuelven PARKING_getNSemAforos() y PARKING_getTamaNoMemoriaCompartida()

(Los usamos aqui:

// obtener tamaño de la memoria compartida y el numero de semaforos

```
    tam = PARKING_getTamaNoMemoriaCompartida();
```

```
    nSem = PARKING_getNSemAforos();)
```

```
char *mem_base = NULL;
```

Puntero al inicio del bloque de memoria compartida.

```
MEM_PROPIA *mp = NULL;
```

Apunta al inicio de la memoria propia del programa, justo después de la zona de la biblioteca. Sirve para acceder al struct de MEM_PROPIA en las funciones que lo necesiten.

4. Creación de Recursos

- Recursos ampliados con + sizeof(MEM_PROPIA) o + NUM_SEM_PROPIOS.

```
id_mem = shmget(IPC_PRIVATE, tam + sizeof(MEM_PROPIA), IPC_CREAT | 0600);
```

```
id_sem = semget(IPC_PRIVATE, nSem + NUM_SEM_PROPIOS, IPC_CREAT | 0600);
```

- Tema de memoria con smath

```
mem_base = (char *)shmat(id_mem, NULL, 0);
```

```
if (mem_base == (char *)-1) {
```

```
    perror("shmat");
```

```
    limpiar();
```

```
    return 1;
```

```
}
```

```
// mp apunta a la zona propia, justo detras de la de la biblioteca
```

```
mp = (MEM_PROPIA *) (mem_base + tam);
```

```
memset(mp, 0, sizeof(MEM_PROPIA));
```

shmget solo crea el segmento de memoria y devuelve un ID numérico. Para poder leer y escribir necesitamos un puntero, que es lo que devuelve shmat. Luego mp se posiciona saltando los tam bytes de la biblioteca, apuntando a la zona de memoria propia (como se cide en el apartado 5). memset inicializa esa zona de memoria propia a cero para evitar problemas como información basura almacenada por error.

- Valor de inicio en proxAparcar

```
for (int i = 0; i < NUM_ALGORITMOS; i++) {  
    mp->proxAparcar[i] = 1;  
}
```

El primer coche que llega tiene número 1, así que todos los algoritmos empiezan con el valor 1 en proxAparcar.

- Semáforos ampliados e inicializados

```
semctl(id_sem, IDX_SEM_CHOFER, SETVAL, 0);  
semctl(id_sem, IDX_SEM_MUTEX, SETVAL, 1);  
for (int i = 0; i < NUM_ALGORITMOS; i++) {  
    semctl(id_sem, IDX_SEM_ORDEN(i), SETVAL, 1);  
}
```

Así se crean todos los semaforos (ante solo se creaban los de la biblioteca). CHOFER en 0 para que los choferes esten bloqueados hasta que se les avise. MUTEX en 1 para que el primero en llegar pueda entrar, y ORDEN en 1 ya que como dijimos antes, el 1 significa que el siguiente proceso se puede liberar (al empezar, el primer coche no tiene a nadie aparcando que lo bloquee).

5. Debug

```
if (debug) {  
    fprintf(stderr, "[MAIN] IPC creados -> SHM:%d SEM:%d MSG:%d\n", id_mem, id_sem,  
id_buzon);  
    // PRUEBA PARA VER SI INICIA CORRECTAMENTE  
    if (resultado == 0) {  
        printf("PARKING_inicio funciona correctamente\n");  
    } else {  
        printf("PARKING_inicio devuelve %d, por tanto falló\n", resultado);  
    }  
}
```

Se añadió un nuevo mensaje de debug para ver que se creen los procesos IPCs a parte de los que había antes.

Ejecutar con D para que se vea, o con `d 2>debug.txt` para guardar el debug en un txt en lugar de verlo por pantalla (solo el stderr). Si se guarda en el txt, los mensajes de `printf` normales no se verán en el txt pero si por pantalla, pero los `printf` con `stderr` se verán solo en el txt y no en pantalla. (Util mas que nada para cuando haya varios, que pueden aparecer en medio de la visual del parking).

6. FUnción Limpiar()

```
void limpiar(void) {
    if (mem_base != NULL && mem_base != (char *)-1) {
        shmdt(mem_base);
    }
    if (id_mem != -1) {
        shmctl(id_mem, IPC_RMID, NULL);
    }
    if (id_sem != -1) {
        semctl(id_sem, 0, IPC_RMID);
    }
    if (id_buzon != -1) {
        msgctl(id_buzon, IPC_RMID, NULL);
    }
} //fin funcion limpiar
```

Básicamente limpia todos los recursos creados. En caso de `id_mem`, `id_sem` e `id_buzon`, se inicializan en -1 de manera que si no se crean, valen -1. Por su parte, `mem_base` no es tan sencillo, si no vale `NULL` (esta vacío) ni `(char *)-1` (esto es un código de error de `shmat` que significa que falló), significa que está creado y por tanto hay que eliminarlo. En caso de valer alguna de las otras dos opciones, no está creado.

Es importante para gestionar a parte de la liberación al acabar el programa, que si la creación de alguno de estos recursos falla, se liberen todos los que fueron creados antes que él.

- **punto 4: Llamar a la función PARKING_inicio en main. Debe aparecer la pantalla de bienvenida y, pasados dos segundos, dibujarse la pantalla. Para el array de funciones de llegada, usad funciones que nada más tengan un pause() dentro, de momento. Todos los procesos que se creen a continuación en sucesivos puntos deben crearse después de haber llamado a PARKING_inicio.**

AÑADIDOS

1. Función de llegada de prueba cambiada por la de cada algoritmo

```
int primer_ajuste(HCoche hc) {  
    pause();  
    return 0;  
} //fin funcion primer_ajuste
```

```
int siguiente_ajuste(HCoche hc) {  
    pause();  
    return 0;  
} //fin funcion siguiente_ajuste
```

```
int mejor_ajuste(HCoche hc) {  
    pause();  
    return 0;  
} //fin funcion mejor_ajuste
```

```
int peor_ajuste(HCoche hc) {
    pause();
    return 0;
} //fin funcion peor_ajuste
```

Con sus prototipos y cambiando lo siguiente:

// array de 4 funciones, una por algoritmo (PRIMER, SIGUIENTE, MEJOR, PEOR)

```
TIPO_FUNCION_LLEGADA funciones[4] = {
    primer_ajuste,
    siguiente_ajuste,
    mejor_ajuste,
    peor_ajuste
};
```

No acabo de entender la idea del pause(), ya que supuestamente para los procesos sin que consuman memoria, pero de momento no tenemos nada, todavía no gestionamos el parking realmente. A estas no se entra todavía porque no tenemos el tema del gestor ni choferes entonces no llega ningún coche, por tanto no llegan a funcionar.

- **punto 5. Podéis ejecutar la práctica, a partir de ahora, redirigiendo la salida de errores a un fichero para que no se os emborrone la pantalla**

AÑADIDOS

1. Mensajes de depuración

Creé mensajes de depuración. Para identificarlos fácilmente para después modificarlos/eliminarlos los puse siguiendo esta estructura:

```
//=====
```

```
if (debug) printf(stderr, "[código-error] MENSAJE DE ERROR");
```

```
//=====
```

Esto con la idea de poder mientras avanzamos identificar problemas, prefiero ir poniéndolos ahora porque después al ir ampliando puede ser que algo de lo que tenemos hecho no funcione bien del todo, así es más sencillo ir comprobando el funcionamiento de todo.

Si se ejecuta el código con ./parking <ret> <chof> <PA oPD si se considera> D, los mensajes aparecen en la propia terminal, por lo que en cuanto haya muchos perderemos la simulación entre tanto mensaje de debug.

La idea es ejecutar el programa con `./parking <ret> <chof> <PA oPD si se considera> D 2>debug.txt` de manera que la simulación vaya por la terminal y los mensajes de debug se guarden en un `.txt`.

Para luego aclararse bien, tenía pensado utilizar códigos identificadores, para saber de qué es el debug en concreto. Por ejemplo, en los que ya hay, uso D-IPC para tema de recursos IPC (creación, limpieza...), D-SIG para tema de señales (de momento no sale ya que no da tiempo de activar la señal) y D-PKG para tema de parking (inicio, final...).

Dejo un cuadro que iré ampliando, según lo que tu o yo vayamos metiendo en este sentido:

CÓDIGO	Mensajes...
D-IPC	...relacionados con recursos IPC
D-SIG	...relacionados con señales
D-PKG	...relacionados con las funciones PARKING de la biblioteca
D-CHOFER	...relacionados con el proceso chofer
D-PADRE	...relacionados con el proceso padre
D-ALG	...relacionados con los algoritmos de ajuste

- **punto 7. Los siguientes pasos conllevan resolver la práctica solo para un algoritmo y con un chófer, para ir, poco a poco, aumentando la dificultad. Para ello, solo programad la función de llegada del primer algoritmo. El resto de funciones de llegada, que devuelvan -2 para indicar a la biblioteca que no queréis, de momento, tratar con ellas. Y el chófer, que solo lea el mensaje y lo imprima en la pantalla.**

1. Funciones de Algoritmos

Todas en return -2 (esto significa que de momento no queremos tener estos algoritmos en funcionamiento). La de primer ajuste (primera a trabajar) en return 0 de momento, antes de añadir logica.

1. Creación de proceso hijo

```
// crear el proceso chofer
pid_t pid_chofer = fork();

if (pid_chofer < 0) {
    // fork fallo, limpiamos y salimos
    perror("fork chofer");
    limpiar();
    exit(1);
}

if (pid_chofer == 0) {
    // ===== PROCESO HIJO (chofer) =====
    // el hijo ignora SIGINT, solo muere cuando el buzón desaparece
    signal(SIGINT, SIG_IGN);
    struct PARKING_mensajeBiblioteca msg;

    while (1) {
        // si el buzón se limpia y msgrcv falla, salimos
        if (msgrcv(id_buzon, &msg, sizeof(msg) - sizeof(long), 0, 0) == -1) {
            break;
        } //fin if

        // imprime lo que ha llegado
        printf("[CHOFER] tipo=%ld subtipo=%ld coche=%d\n", msg.tipo, msg.subtipo,
msg.hCoche);
    }
}
```

La lógica por partes es la siguiente:

pid_t pid_chofer = fork();

→ Crea el proceso hijo y devuelve el PID al padre. Esto crea 1 chofer, para luego hay que hacer un bucle que cree los choferes que se pidan como argumento

if (pid_chofer < 0)

→ Si es <0 significa que hubo un error al crear el hijo

if (pid_chofer == 0)

→ Si es 0 se creo correctamente y el proceso hijo entra en el if

signal(SIGINT, SIG_IGN);

→ El proceso hijo ignora la señal ctrl+c. La idea es que mueran como se explica más abajo, cuando los recursos están borrados e intentan acceder a ellos. Si dejásemos todos con la señal activa, pensemos en un proceso padre, todos los recursos se crean y buzón = 0 (pensando solo en el buzón, pero lo mismo para los demás recursos IPC). Si ahora se crea un hijo, para el hijo buzón = 0 (lo hereda del padre). Por tanto, cuando luego se entre en limpiar(), el padre comprueba, buzón != -1? si, pues limpio recursos. El hijo entra, pero el sigue pensando que buzón = 0 (no se transmite el cambio de esa limpieza hacia el hijo), por tanto buzón != -1? si, pues limpio recursos. problema? esos recursos ya no existen, por que ya estaban borrados, por tanto da error. Lo mismo con los demás hijos que pueda haber

struct PARKING_mensajeBiblioteca msg;

→ Variable donde se guarda el mensaje recibido, está definida en parking.h. Es un struct que contiene:

- **long tipo:** Identifica el mensaje, de donde viene. Normalmente (no se si siempre) de Parking.
- **long subtipo:** Puede valer PARKING_MSGSUB_APARCAR(1) o PARKING_MSGSUB_DESAPARCAR(2). Indica si el mensaje pide gestionar un aparcamiento o un desaparcamiento.
- **HCoche hCoche:** Numero del coche al que pertenece el mensaje.

msgrcv(id_buzon, &msg, sizeof(msg) - sizeof(long), 0, 0);

→ Esto contiene lo siguiente:

- **id_buzon:** Identificador del buzón que msgget() nos dio al crearlo.
- **&msg:** Es un puntero a la variable donde el kernel va a copiar el mensaje del buzón (es importante entender que, el buzón solo es una cola con mensajes y que kernel

es quien la gestiona). La variable es de tipo structPARKING_mensajeBiblioteca (el definido justo arriba). Siempre es la misma posición de memoria, el mensaje se va sustituyendo.

- **sizeof(msg) - sizeof(long):** msgrcv exige el tamaño del cuerpo del mensaje, sin contar el campo tipo. (Esto es una convención histórica de System V IPC: el campo tipo es siempre un long y va fuera del cuerpo.):

sizeof(msg) → tamaño total de la estructura (tipo + subtipo + hCoche)

sizeof(long) → tamaño del campo tipo

sizeof(msg) - sizeof(long) → tamaño del cuerpo (subtipo + hCoche) ← esto es lo que quiere msgrcv

- **Primer 0:** Controla el tipo de mensaje que queremos recibir. 0 nos indica que queremos el primer mensaje de la cola.
- **Segundo 0:** Flag de comportamiento. 0 nos dice que si la cola está vacía, el proceso se duerme hasta que llegue algo. Así nos ayudamos a no consumir CPU al no estar todo el rato en bucle comprobando si hay o no hay mensajes.

if (msgrcv(.....) == -1)

→ msgrcv intenta leer el mensaje del buzón, si esta acción devuelve -1 significa que hubo un error. Aquí es cuando se mata al proceso hijo al salir del bucle con el exit(0). No es exit(1) ya que aunque haya un error (msgrcv devuelve -1 por un error) es el comportamiento correcto, en el momento en que el padre limpia recursos, los hijos deben de morir de manera ordenada. (aquí creo que habría que añadir un wait() después de limpiar para que el padre no muera antes que los hijos, ya que si no serían zombies y creo que así no se limpiaron correctamente, pero de momento lo dejo así).

printf("[CHOFER] tipo=%ld subtipo=%ld coche=%d\n", msg.tipo, msg.subtipo, msg.hCoche);

→ Print que pide dio, para comprobar si el chofer lee correctamente el mensaje.

- punto 9. Que el chófer distinga entre los dos tipos de mensajes y llame a PARKING_aparcar o PARKING_desaparcar, según corresponda. Ya deberíais ser capaces de ver a los coches aparcar y salir, cuando toque.

1. If que diferencia entre aparcar o desaparcar

Cambié el código del while, ahora dentro tiene esto:

```
if (msg.subtipo == PARKING_MSGSUB_APARCAR) {
    //=====
    if (debug) fprintf(stderr, "[D-CHOFER: aparcar] PID=%d -> Coche %d va
a aparcar\n", getpid(), msg.hCoche);
    //=====
    PARKING_aparcar(
        msg.hCoche,
        &alg_aux,
        aparcar_commit,
        permiso_avance,
        permiso_avance_commit
    );
} else if (msg.subtipo == PARKING_MSGSUB_DESAPARCAR) {
    //=====
    if (debug) fprintf(stderr, "[D-CHOFER: desaparcar] PID=%d -> Coche
%d va a desaparcar\n", getpid(), msg.hCoche);
    //=====
    PARKING_desaparcar(
        msg.hCoche,
        &alg_aux,
        permiso_avance,
        permiso_avance_commit
    );
} //fin else if
```

Como dijimos en su momento el mensaje que recibe el chofer tiene un campo subtipo, este es el que dice si el coche quiere aparcar o desapparcar. Si es PARKING_MSGSUB_APARCAR llamamos a PARKING_aparcar y si es PARKING_MSGSUB_DESAPARCAR a PARKING_desapparcar

- **punto X. Aqui ire poniendo todo lo que hago a partir de ahora y lo iré explicando, ya que de ahora en adelante no hay un “orden” si no que hay que ir haciendo todo lo que falta a los pocos (mejor si esto lo revisas con el código al lado, que aquí lo tengo fragmentado por partes)**

1. Función primer_ajuste

La función hace lo siguiente:

```
int longitud = PARKING_getLongitud(hc);
```

```
int huecoLibre = 0;
```

```
int pos = -1;
```

→ Básicamente, obtiene la longitud del coche, huecoLibre se inicializa en 0 (es el contador de huecos libres consecutivos, que permite saber si el coche entra en un hueco) y pos es la variable que nos dice en qué posición del parking debe estar el culo del coche (si el coche aparca de la posición 0 a la 4, valdrá 0 y se colocará ahí el comienzo del coche), si vale -1 es que no encontró hueco.

```
for (int i = 0; i < TAM_PARKING; ) {
```

→ Recorre el parking de inicio a fin

```
    if (mp->acera[PRIMER_AJUSTE][i] == 0) {  
        huecoLibre++;  
        if (huecoLibre >= longitud) {  
            pos = i - longitud + 1;  
            break;  
        }  
    }
```

```

        i++;
    } else {
        i += mp->acera[PRIMER_AJUSTE][i];
        huecoLibre = 0;
    }
}

```

→ Aquí, el if del principio comprueba si la posición de la acera en el primer ajuste en la posición i dentro del parking ($acera[PRIMER_AJUSTE][i]$) está libre (0 significa libre). Si lo está suma uno a `huecoLibre`. Después, si ese `huecoLibre` es igual o mayor que la longitud del coche este cabe en el hueco. Como puede aparcar se calcula el índice donde deberá empezar a aparcar (donde tiene que ir el culo del coche) y se sale del bucle for (ya encontramos el primer hueco disponible). En caso de no entrar al if de $huecoLibre \geq longitud$, y se sumaría en 1 (avanzamos una posición dentro del parking).

Si el if del principio no se cumple, y el valor de de la acera de primer ajuste para la posición i no vale 0, está ocupada la posición, por tanto `huecoLibre` pasa a valer 0 (para medir bien el siguiente hueco) e i salta el número que tenga la acera en esa posición (la idea de esto es que si hay un coche de 11 de largo, $acera[PRIMER_AJUSTE][i]$ guarda ese valor (11), por tanto, sabemos que ese número es la cantidad de posiciones que ocupa el coche, por lo que sería una tontería comprobar todas las posiciones intermedias entre i e $i+11$ porque van a estar ocupadas, no se hasta que punto marca diferencia pero así está algo más optimizado).

EJEMPLO VISUAL:

pos: 0 1 2 3 4 5

0 0 1 0 0 0

→ $i=2$ está ocupado →

pos: 0 1 2 3 4 5

0 0 1 0 0 0

$i=3 \rightarrow huecoLibre=1$

$i=4 \rightarrow huecoLibre=2$

$pos = i - longitud + 1;$

$i=5 \rightarrow huecoLibre=3 \geq longitud(3) \rightarrow inicio = 5 - 3 + 1 = 3 \rightarrow break$

pos: 0 1 2 3 4 5

0 0 1 3 3 3

```

if (pos >= 0) { // si encuentro hueco

```

```

        for (int i = pos; i < pos + longitud; i++) { // recorre las posiciones que ocupara el
coche

                mp->acera[PRIMER_AJUSTE][i] = longitud; // las marca como ocupadas (las
marca con el numero de longitud del coche)

        }

}

return pos;

```

→ Si $pos \geq 0$ significa que encontró hueco (recordemos que -1 era que no lo encontró), entonces, recorre las posiciones que ocupa el coche con el bucle for, desde el valor de pos (donde va el culo del coche) hasta que i sea pos+longitud (ya se habrán guardado todos los espacios que ocupa el coche). y guarda el valor de la longitud del coche como dijimos antes. Aquí es donde realmente queda registrado que el coche aparcó. Luego devuelve pos que es el valor que necesita la biblioteca.

2. Función sem_esperar_turno

La función hace lo siguiente:

```
void sem_esperar_turno(int idx_sem, int num_coche) {
```

→ Recibe el índice del semáforo y el número de coche que quiere aparcar.

```
struct sembuf op;
```

→ Declaración de la estructura de operación (op) que le dirá al kernel que es lo que tiene que hacer.

```
op.sem_num = idx_sem;
```

→ Dice al kernel sobre qué semáforo del array vamos a trabajar. Recordemos que hay un semáforo de orden por algoritmo, `IDX_SEM_ORDEN[0]` para primer ajuste y así con los demás hasta `IDX_SEM_ORDEN[3]`.

```
op.sem_op = -num_coche;
```

→ La operación que hará el kernel. La idea es la siguiente. `op.sem_op` (abreviatura de semaphore operation) resta al valor del semáforo el número del coche que quiere aparcar. Es la operación que dictamina si el coche debe esperar turno o si ya le toca a él.

EJEMPLO VISUAL:

Los semáforos de orden se inicializan a 1, por tanto pensemos que acaba de empezar el programa y ya llegan 3 coches.

Valor del semaforo = 1, entran los 3 coches:

$$X = (\text{op.sem_op} = -\text{num_coche}) = \text{numero del semaforo} - \text{numero coche} = 1 - 1 = 0 \leftarrow$$

$$X = (\text{op.sem_op} = -\text{num_coche}) = \text{numero del semaforo} - \text{numero coche} = 1 - 2 = -1$$

$$X = (\text{op.sem_op} = -\text{num_coche}) = \text{numero del semaforo} - \text{numero coche} = 1 - 3 = -2$$

Valor del semaforo = 2, entran los 2 coches que quedan:

$$X = (\text{op.sem_op} = -\text{num_coche}) = \text{numero del semaforo} - \text{numero coche} = 2 - 2 = 0 \leftarrow$$

$$X = (\text{op.sem_op} = -\text{num_coche}) = \text{numero del semaforo} - \text{numero coche} = 2 - 3 = -1$$

Los coches que “pasan” son los marcados con $\leftarrow$, los demás quedan esperando turno sin consumir CPU. La condición para “pasar” y aparcar y no quedarse en espera de turno viene programada por el propio kernel, y es que X sea 0. (la condicion real de kernel es que numero de semaforo sea $\geq$ numero coche, pero para nuestro caso podemos verlo como lo planteé arriba, que es más sencillo de ver y es como va a ocurrir en la practica)

`op.sem_flg = 0;`

→ Sirve para pautar comportamientos especiales, el 0 es el normal, el que hay comentado en el ejemplo visual de arriba.

`semop(id_sem, &op, 1);`

→ Aquí es donde realmente le decimos a kernel que haga la operación. Si X no es 0, el proceso se duerme aquí. Solo se despertará cuando sem_liberar_turno cambie el semáforo a un valor suficiente. El 1 indica que solo hay una operación, que es la resta.

`op.sem_op = +num_coche;`

`semop(id_sem, &op, 1);`

→ Cuando semop hace el calculo de la resta, el semáforo se queda con el valor X (por ejemplo, 0 si el coche estaba en su turno). El problema de esto son dos:

- Si el semáforo tuvo que esperar por su turno, calculó $X = -1$ (por ejemplo, podría ser -2 o cualquier valor negativo), y esto hará que cuando se vuelva a comprobar si tiene ya turno o si debe seguir esperando, el semáforo tenga valor -1, por tanto valor de semáforo - número de coche (pongamos que va a aparcar el 2) sería $-1 - 2 = -3$, después $-3 - 3 = -6...$ y empezamos un bucle en el que nunca tomaría el turno y cada vez tendría un valor más pequeño
- Si el semaforo da permiso para tomar el turno, después en aparcar_commit aparece sem_liberar_turno que usa SETVAL para setear al semáforo el valor de semaforo +1.

3. Función sem_liberar_turno

```
semctl(id_sem, idx_sem, SETVAL, num_coche + 1);
```

→ Usa SETVAL para poner el semáforo al número del siguiente coche. De esta forma aseguramos que el siguiente que tome el turno sea el coche que sigue numéricamente al último que aparcó.

4. Función permiso_avance

```
int X1 = PARKING_getX(hc);
```

```
int Y1 = PARKING_getY(hc);
```

```
int X2 = PARKING_getX2(hc);
```

```
int Y2 = PARKING_getY2(hc);
```

```
int alg = PARKING_getAlgoritmo(hc);
```

→ Tomamos los valores que usaremos más adelante

```
if (Y1 == 2 && Y2 == 2 && X2 >= 0 && X2 < TAM_PARKING)
```

→ Si Y1 e Y2 son 2 es que nos movemos por el carril

```
struct sembuf op;
```

→ Declaración de la estructura de operación (op) que le dirá a kernel que es lo que tiene que hacer.

```
while (1)
```

→ Bucle que se va a repetir hasta que el coche consiga ocupar su siguiente posición. Si no puede avanzar, se dormirá dentro y lo volverá a intentar al despertar.

```
op = (struct sembuf){IDX_SEM_MUTEX, -1, 0};
```

```
semop(id_sem, &op, 1);
```

→ Esto aunque parezca algo raro, es como en la función sem_esperar_turno que es similar a esta, en la que se hace:

```
op.sem_num = idx_sem;
```

```
op.sem_op = -num_coche;
```

```
op.sem_flg = 0;
```

Esto es lo mismo pero más compactado, seria como hacer esto:

```
op.sem_num = IDX_SEM_MUTEX;
```

→ Dice al kernel sobre qué semáforo del array vamos a trabajar. En este caso `IDX_SEM_MUTEX`, que es el semáforo de exclusión mutua para que no puedan dos procesos acceder al mismo tiempo a la memoria compartida. Si dos coches acceden a la vez a la matriz de posiciones del carril, podrían dos coches ver una posición a la que quieren avanzar libre, ambos moverse y colisionar. Se necesita un orden en ese acceso, por tanto, todo el tema de accesos que viene a partir de ahora para comprobar ocupaciones y liberaciones de posición, siempre serán controlados por este semáforo.

```
op.sem_op = -1;
```

→ La operación que hará el kernel. Resta 1 al valor del semáforo. Si el resultado es negativo (es decir, si el semáforo mutex vale 0), el proceso se duerme hasta que otro proceso lo libere.

```
op.sem_flg = 0;
```

→ Sirve para pautar comportamientos especiales, el 0 es el normal, el que hay comentado en el ejemplo visual de arriba.

Luego, `semop(id_sem, &op, 1)`; es donde realmente le decimos al kernel que ejecute la operación y duerme el proceso si el resultado es negativo.

```
if (mp->carril[alg][X2] == 0) {  
    mp->carril[alg][X2] = 1;  
    op = (struct sembuf){IDX_SEM_MUTEX, +1, 0};  
    semop(id_sem, &op, 1);  
    break;  
}
```

→ Comprueba si la posición X2 del carril está libre (0 = libre, 1 = ocupado). Si lo está, la ocupa marcándola como 1. Esto todo controlado por el mutex. En `op = (struct sembuf){IDX_SEM_MUTEX, +1, 0}; semop(id_sem, &op, 1);` como ya explicamos un poco [más arriba](#).

Algo similar pero esta vez la operación hace +1, ya que como la posición X2 estaba libre pudo ocuparla, entonces libera el mutex para que otro coche pueda acceder a la matriz de posiciones del carril y avanzar.

(En mutex, operación -1 busca ocupar el semáforo, y +1 liberarlo, la gracia es que si un proceso toma el mutex, que empieza valiendo 1, le hace -1, y como $1-1 = 0$, entra y ahora `mutex = 0`, por tanto el siguiente proceso hace -1 y como ahora vale 0, $0-1$ da negativo se duerme el proceso, y así todos los procesos que quieran avanzar. Después, cuando el que entró en el mutex ocupó una posición, hace +1 y ahora el semáforo mutex vale 1, por tanto el proceso que antes estaba dormido (uno de ellos) ahora puede hacer -1 al semáforo, y como $1-1 = 0$, entra. Así se van pasando turnos).

El `break` sale del bucle porque consiguió avanzar.

```
op = (struct sembuf){IDX_SEM_MUTEX, +1, 0};
```

```
semop(id_sem, &op, 1);
```

→ Si el proceso no pudo ocupar la posición porque estaba ocupada por otro proceso, libera el mutex para que otro proceso pueda avanzar. Si yo no puedo avanzar, pues que otro lo intente.

```
op = (struct sembuf){IDX_SEM_AVANCE(alg), -1, 0};
```

```
semop(id_sem, &op, 1);
```

→ Aquí es donde el proceso se duerme si no pudo ocupar la posición porque estaba ocupada por otro coche. Para despertarse, en `permiso_avance_commit`, es donde se hace el +1, ya que el commit es donde se libera la posición que desocupó el coche y por tanto solo se llega a ella tras un avance, por lo que debe liberarse `sem_avance` para que otro que no pudo avanzar antes pueda intentarlo otra vez.

Es importante diferenciar entre `IDX_SEM_MUTEX` y `IDX_SEM_AVANCE`.

→ **MUTEX** bloquea a procesos si hay uno accediendo a la memoria compartida, ya sea para comprobar si la posición a la que quiere avanzar está libre y ocuparla, o desocupar la posición que abandonó al avanzar. Cuando el proceso lee y modifica si es necesario la matriz correspondiente (la acera o el carril) en la memoria compartida, se libera el semáforo `MUTEX` para que otro proceso pueda acceder a la memoria compartida. La idea aquí, es dormir a los procesos mientras esperan a poder acceder a esa memoria compartida.

→ **AVANCE** bloquea procesos que ya comprobaron si podían avanzar, es decir, ya pasaron por el mutex, pero que al intentarlo tenían otro proceso ocupando la posición a la que querían avanzar. Se duermen hasta que otro proceso avanza correctamente y libera la posición que abandonó (esto se hace en `permiso_avance_commit`), entonces los procesos se despiertan para volver a comprobar si ahora si que pueden avanzar (si el coche que les bloqueaba es el que avanzó). Para poder comprobarlo, necesitan poder acceder a la memoria compartida y, por tanto, volver a pasar el filtro del `MUTEX`.